

Key Design Decisions

Decision 1 Neo4j Why we used these is because main queries are basically "find all items that share ingredients across cuisines" which is a graph traversal problem. Doing that in normal relational database database SQL needs like 3 self-joins on a 151k row table and it gets really slow. With Neo4j each PAIRS_WITH edge is already precomputed so the query is just one hop. Looked at Postgres with a co-occurrence table but it just recreates the graph structure manually, which felt like extra work for no real benefit

Decision 2. Precomputing PAIRS_WITH edges | If you calculate ingredient co-occurrence at query time the agent has to scan all 150k item-ingredient links on every single request. That takes way too long for a chat-style agent. So compute it once at load time (takes about a minute) and store the `frequency` and `cuisines` directly on the edge so runtime queries are basically instant. | Tried doing it on the fly first, queries were taking 10+ seconds, not really usable

Decision 3 GPT-4o at temperature=0 for Cypher generation Cypher is not creative writing, it's a precise translation task. I want the same question to always produce the same query, especially during evals. Setting temp=0 makes it deterministic. Tried GPT-4o-mini to save cost but it was making way more syntax errors on multi-hop queries so had to switch back

Decision 4 Showing the LLM its own failed Cypher when retrying When a query fails I send back the exact query that broke plus the Neo4j error message. Turns out the model is pretty good at fixing its own mistakes when you show it what went wrong, way better than just asking it to try again from scratch. Regenerating from scratch on failure worked maybe 40% of the time, showing the error context bumps that to around 85% |

Decision 5 Ground-truth assertions in eval + LLM judge Only using an LLM to score accuracy is kind of easy to game - a confident sounding wrong answer can still score well. So for accuracy I wrote actual Python assertions that check the raw database rows (like "must have 5+ rows" or "must contain a numeric column"). The LLM judge only handles relevance and creativity where there's no single right answer. | Pure LLM judge felt risky, keyword matching was too brittle for open ended questions

Decision 6 APOC for computing pairs The original approach was a Python loop that sent batches to Neo4j one at a time. Works fine but there's a lot of network back and forth. With `apoc.periodic.iterate` the whole MATCH+MERGE runs server side, no round trips per batch. Went from a couple minutes down to under 10 seconds for 9k relationships. |

Python loop batching worked correctly but was noticeably slower, switched once I saw APOC was already available via docker-compose

Decision 7 Batching with UNWIND (5000 rows per batch) | Sending 151k rows one at a time would take ages. Batching into 5k row chunks means only 31 trips to Neo4j instead of 151,000. Makes the full load take 2-3 mins which is pretty reasonable. Started with 500 rows per batch, bumped to 5000 after testing and it made a noticeable difference

Decision 8 Chain as its own node type Chains like Shake Shack have dozens of locations in the data. If I just store chain_name as a text property on Restaurant then comparing chain vs independent restaurants becomes a string match across the whole table. As a proper node you can just traverse PART_OF_CHAIN which is much cleaner. Storing it as a property on Restaurant worked for basic lookups but aggregation queries got messy

Decision 9 FlavorProfile nodes The CSVs have no flavor data at all, just ingredient names. I wanted the agent to answer things like "what smoky ingredients are trending" without relying on keyword matching. So I built a mapping (cumin = spicy + savory, vanilla = sweet, etc) and loaded them as nodes with HAS_FLAVOR edges. Its not perfect but it works well for most questions | Could have put flavor tags as a list property on Ingredient nodes but then you lose the graph structure and traversal queries dont work as cleanly

Decision 10 Distinctivness ratio for cuisine queries | When you ask "what spices are common in Indian cuisine" and just sort by raw count you get things like salt and pepper at the top because they appear everywhere. Sorting by ``indian_uses / total_uses`` gives you turmeric, coriander, garam masala - stuff thats actually specific to Indian food. ORDER BY raw count gives misleading results when restaurants serve multiple cuisines